KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT
24MEC51

EXERCISE NO: 1

GURUTHARSAN S - 24MER011

HARISH KUMAR K K - 24MER015

MADHANRAJ K - 24MER028

SADAAGOPAAN T R - 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 1

TITLE:

Design and develop a Water Level Detection and Flood Alert System using Ultrasonic Sensor. The system measures water levels in rivers, tanks, or dams and provides early warning alerts during overflow conditions.

Done By:

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

Project Overview:

The Water Level Detection and Flood Alert System using an Ultrasonic Sensor is a safety and monitoring system designed to measure the water level in tanks, rivers, canals, dams, and other water-storage or water-flow environments. The system uses an ultrasonic sensor to determine the distance between the sensor and the water surface without requiring direct contact with the water.

The measured distance is processed by a microcontroller such as an Arduino Uno. Based on predefined water-level limits, the system classifies the condition as Normal, Warning, or Critical. LEDs and a buzzer provide immediate visual and audible alerts when the water level approaches or exceeds the danger limit.

This prototype demonstrates a low-cost method of continuous water-level monitoring and early flood/overflow warning. The system can later be extended with GSM, Wi-Fi, IoT dashboards, solar power, and cloud-based monitoring.

Problem Statement

Flooding and water overflow can cause serious damage to human life, agricultural land, infrastructure, vehicles, and property. Conventional water-level monitoring methods often depend on manual inspection or simple mechanical indicators.

These methods have several limitations:

- Continuous monitoring is difficult.
- Manual observation requires human presence.
- Sudden increases in water level may not be detected quickly.
- Conventional systems may not provide immediate warning.

- Direct-contact sensors can suffer from corrosion and contamination.
- Remote locations such as dams and rivers are difficult to monitor manually.

Therefore, there is a need for a low-cost, non-contact, automatic water-level monitoring system that can continuously measure water level and generate an early warning when the water reaches a dangerous level.

Proposed Solution

The proposed system uses an HC-SR04 ultrasonic sensor, Arduino Uno, LEDs, and a buzzer.

The ultrasonic sensor is installed above the water surface. It sends ultrasonic waves toward the water and receives the reflected waves. The Arduino calculates the distance between the sensor and the water surface using the time taken by the ultrasonic wave to return.

The water level is calculated using:

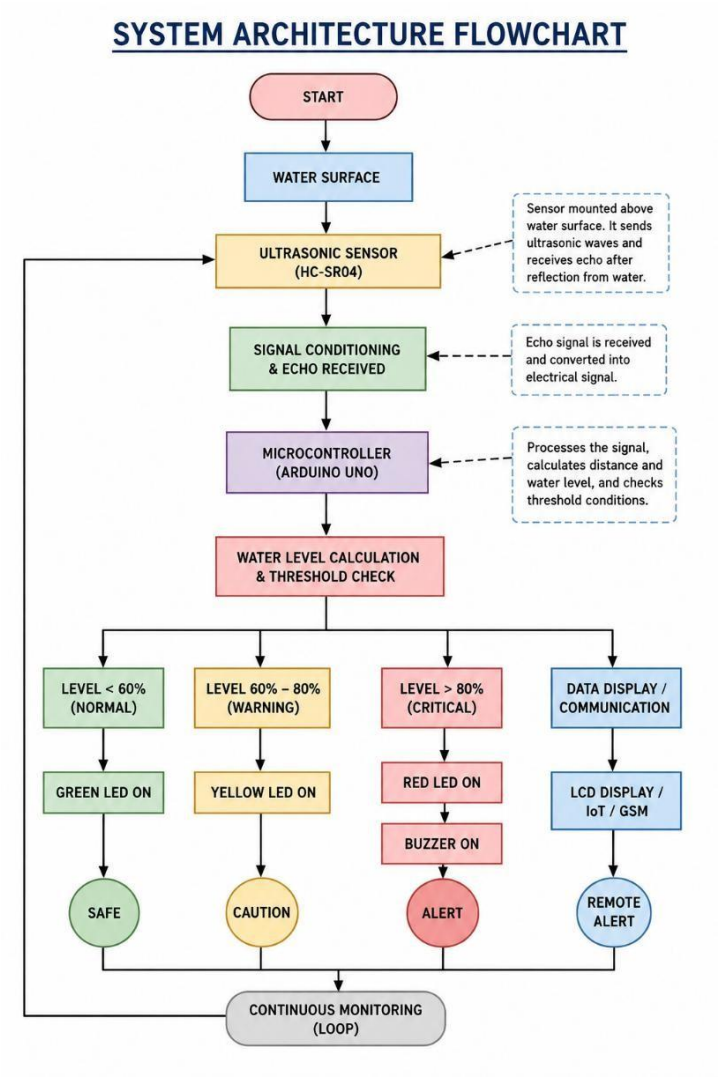
Water Level = Tank/Channel Height − Measured Distance

The system uses three conditions:

Water Condition	Water Level	Indication
Normal	Below 60%	Green LED
Warning	60–80%	Yellow LED
Critical	Above 80%	Red LED + Buzzer

When the water level reaches the critical limit, the buzzer is activated to provide an immediate flood/overflow warning.

System Architecture

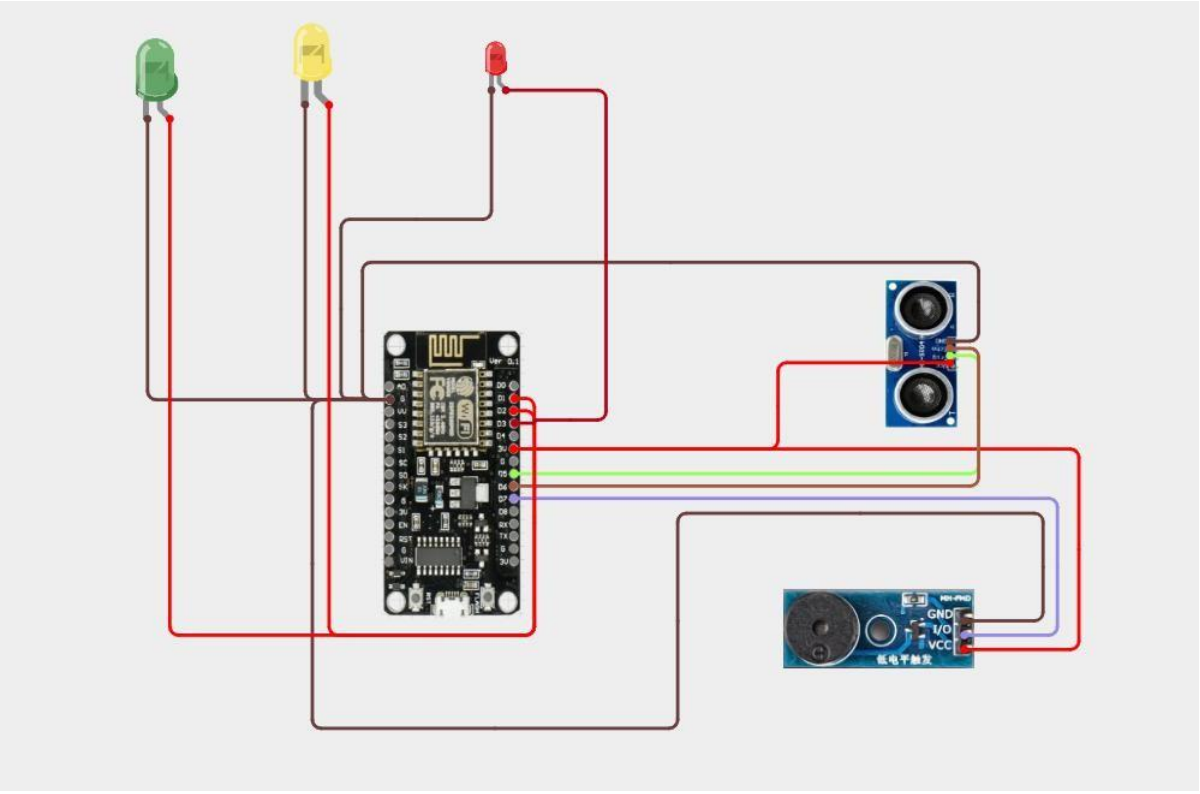


Circuit Table

Component	Pin/Terminal	Arduino Uno Connection
HC-SR04	VCC	5V
HC-SR04	GND	GND
HC-SR04	TRIG	Digital Pin 9
HC-SR04	ECHO	Digital Pin 10
Green LED	Anode	Digital Pin 2 through 220 Ω resistor

Green LED	Cathode	GND
Yellow LED	Anode	Digital Pin 3 through 220 Ω resistor
Yellow LED	Cathode	GND
Red LED	Anode	Digital Pin 4 through 220 Ω resistor
Red LED	Cathode	GND
Buzzer	Positive	Digital Pin 5
Buzzer	Negative	GND
Arduino	5V	Sensor/Component supply
Arduino	GND	Common ground

Circuit Design



Algorithm

1. START

2. Initialize Arduino Uno.

3. Initialize HC-SR04 Ultrasonic Sensor.

4. Initialize Green LED, Yellow LED, Red LED and Buzzer.

5. Set the total tank/channel height.

6. Send a trigger pulse from the ultrasonic sensor.

7. Receive the reflected ultrasonic signal through the ECHO pin.

8. Measure the time taken by the ultrasonic wave to return.

9. Calculate the distance between the sensor and water surface.

$$\text{Distance} = (\text{Echo Time} \times 0.0343) / 2$$

10. Calculate the water level.

$$\text{Water Level} = \text{Tank Height} - \text{Measured Distance}$$

11. Calculate the water level percentage.

Water Level % = (Water Level / Tank Height) × 100

12. Compare the water level percentage with the predefined limits.

13. If Water Level < 60%:

Turn ON Green LED.

Display "NORMAL".

14. Else if Water Level ≥ 60% and ≤ 80%:

Turn ON Yellow LED.

Display "WARNING".

15. Else if Water Level > 80%:

Turn ON Red LED.

Turn ON Buzzer.

Display "CRITICAL – FLOOD ALERT".

16. Display the measured distance and water level.

17. Repeat the process continuously for real-time monitoring.

18. STOP

Coding

```
/*****************
 Water Level Detection & Flood Alert System
 NodeMCU ESP8266
*****************/

#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"

// WiFi Credentials
#define WIFI_SSID "YOUR_WIFI_NAME"
#define WIFI_PASS "YOUR_WIFI_PASSWORD"

// Adafruit IO Credentials
#define IO_USERNAME "YOUR_ADAFRUIT_USERNAME"
#define IO_KEY "YOUR_ADAFRUIT_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,
WIFI_PASS);

// Adafruit IO Feeds
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *distanceFeed = io.feed("distance");
AdafruitIO_Feed *statusFeed = io.feed("status");
```

```
// GPIO Pins
#define TRIG_PIN    14   // GPIO14 (D5)
#define ECHO_PIN    12   // GPIO12 (D6)

#define GREEN_LED   5   // GPIO5 (D1)
#define YELLOW_LED  4   // GPIO4 (D2)
#define RED_LED     13   // GPIO13 (D7)

#define BUZZER      15   // GPIO15 (D8)

// Tank Height
const float tankHeight = 100.0;   // cm

bool alertSent = false;
unsigned long lastUpload = 0;

void setup() {

  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
```

```
  pinMode(GREEN_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Adafruit IO Connected!");
}

void loop() {
```

```
io.run();

float distance = readDistance();
float level = ((tankHeight - distance) / tankHeight) * 100.0;
level = constrain(level, 0, 100);

String status = "SAFE";

if (level < 40) {

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  status = "SAFE";
  alertSent = false;
}

else if (level < 80) {

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, HIGH);
```

```
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER, LOW);

  status = "WARNING";
  alertSent = false;
}

else {

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  digitalWrite(BUZZER, HIGH);

  status = "DANGER";

  if (!alertSent) {
    Serial.println("SMS Sent: Danger Alert");
    alertSent = true;
  }
}

// Serial Monitor
Serial.print("Distance: ");
```

```
  Serial.print(distance, 1);
  Serial.print(" cm | Level: ");
  Serial.print(level, 1);
  Serial.print("% | Status: ");
  Serial.println(status);

  // Upload to Adafruit IO every 5 seconds
  if (millis() - lastUpload >= 5000) {

    waterLevelFeed->save(level);
    distanceFeed->save(distance);
    statusFeed->save(status);

    lastUpload = millis();
  }

  delay(300);
}

// Ultrasonic Function
float readDistance() {

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
```

```
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0)
    return tankHeight;

  return duration * 0.0343 / 2.0;
}  Serial.println(status);

  delay(1000);
}
```

System Working

The system operates continuously to monitor the water level.

Initially, the ultrasonic sensor is placed above the water surface. The Arduino sends a trigger pulse to the HC-SR04 sensor. The sensor transmits ultrasonic waves toward the water surface.

When the ultrasonic wave hits the water surface, it is reflected back toward the sensor. The sensor generates an echo signal corresponding to the travel time of the ultrasonic wave.

The Arduino uses this time to calculate the distance between the sensor and water surface. Since the sensor is installed at a known height, the actual water level can be calculated.

For example, if the tank height is 30 cm and the measured distance from the sensor to the water surface is 6 cm:

Water Level = 30 − 6

Water Level = 24 cm

Therefore:

Water Level Percentage = (24 / 30) × 100

Water Level Percentage = 80%

At this point, the system enters the critical condition and activates the red LED and buzzer.

Bills Of Material

S.No.	Component	Quantity	Purpose
1	Arduino Uno	1	Main Controller
2	HC-SR04 Ultrasonic Sensor	1	Water Level Detection
3	Green LED	1	Normal Level Indication
4	Yellow LED	1	Warning Indication
5	Red LED	1	Critical Level Indication
6	220 Ω Resistor	3	LED Protection
7	Buzzer	1	Flood Alert
8	Breadboard	1	Circuit Assembly
9	Jumper Wires	1 Set	Electrical Connections
10	5V USB Power Supply	1	Power Supply
11	Water Container	1	Prototype Tank

Prototype Implementation

The prototype can be constructed using a small transparent plastic container to represent a tank or water-storage structure.

Implementation Steps

Step 1 – Prepare the tank

A transparent container is used to visually observe the water level.

Step 2 – Mount the ultrasonic sensor

The HC-SR04 sensor is fixed at the top of the container facing downward toward the water surface.

Step 3 – Connect the Arduino

The TRIG and ECHO pins are connected to the Arduino according to the circuit table.

Step 4 – Connect the indicators

Green, yellow, and red LEDs are connected to the Arduino to indicate different water-level conditions.

Step 5 – Connect the buzzer

The buzzer is connected to the Arduino and programmed to activate during the critical water-level condition.

Step 6 – Upload the program

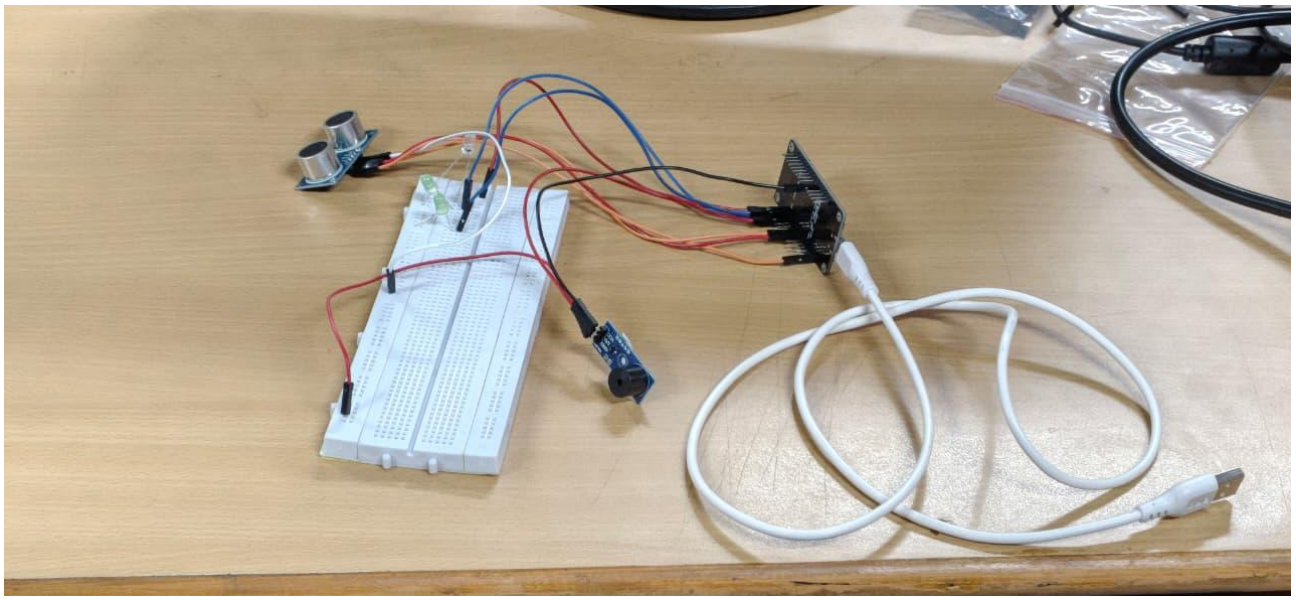
The Arduino program is uploaded using the Arduino IDE.

Step 7 – Test different water levels

Water is gradually added to the container. The ultrasonic sensor continuously measures the distance to the water surface.

Step 8 – Observe alerts

The LED indication changes according to the water level. When the water reaches the critical threshold, the red LED and buzzer are activated.



Results & Performance Analysis

The prototype can be tested by gradually increasing the water level inside the container.

Sample Test Results

Test	Water Level	Level %	System Condition	Output
1	5 cm	16.7%	Normal	Green LED
2	10 cm	33.3%	Normal	Green LED
3	15 cm	50%	Normal	Green LED
4	18 cm	60%	Warning	Yellow LED
5	21 cm	70%	Warning	Yellow LED
6	24 cm	80%	Critical	Red LED + Buzzer
7	27 cm	90%	Critical	Red LED + Buzzer
8	29 cm	96.7%	Critical	Red LED + Buzzer

Performance Analysis

The prototype provides continuous, non-contact measurement of water level. The ultrasonic sensor eliminates the need for physical contact between the sensing element and water.

The system provides three levels of indication:

- Green: Safe water level
- Yellow: Water level requires attention

Red + Buzzer: Critical water level and early flood/overflow warning

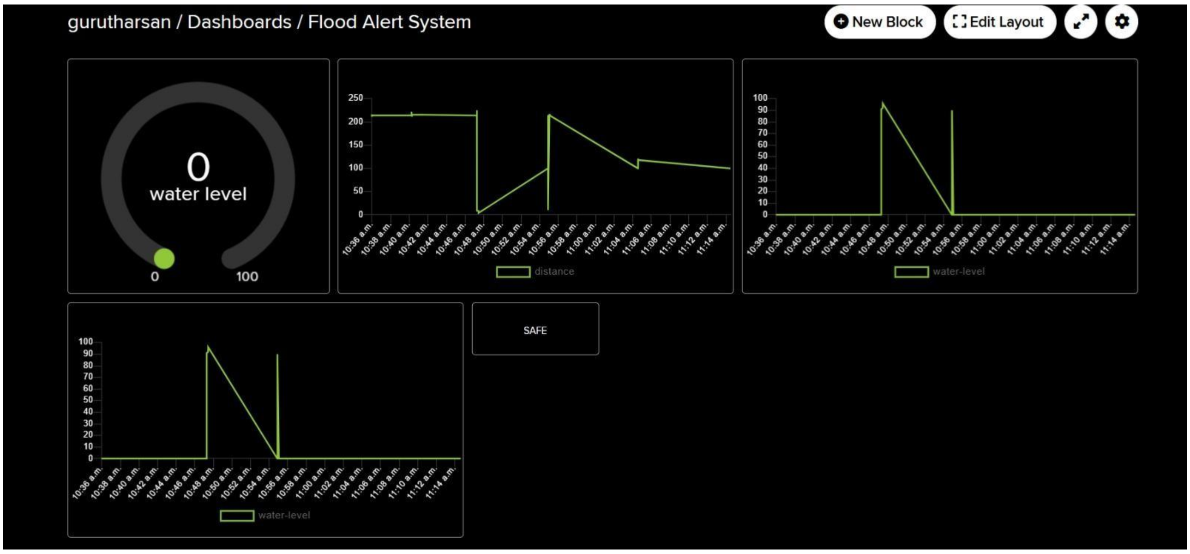
The system is inexpensive, simple to construct, and suitable for educational prototypes and small-scale water-level monitoring applications.

The accuracy of the system depends on several factors:

- Ultrasonic sensor accuracy
- Sensor mounting position
- Water-surface turbulence
- Temperature and environmental conditions
- Sensor alignment
- Electrical noise
- Tank geometry

For a practical flood-monitoring system, multiple readings can be averaged to reduce fluctuations caused by waves and turbulence.

```
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
Distance: 100.0 cm | Level: 0.0% | Status: SAFE
```



```
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
Distance: 0.5 cm | Level: 99.5% | Status: DANGER
```

```
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 100.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
Distance: 398.00 cm | Level: 0.00% | SAFE
Distance: 399.00 cm | Level: 0.00% | SAFE
Sending SMS...
SMS Sent.
Distance: 7.00 cm | Level: 93.00% | DANGER
Distance: 10.00 cm | Level: 90.00% | DANGER
Distance: 100.00 cm | Level: 0.00% | SAFE
```

```
)istance: 399.00 cm | Level: 0.00% | SAFE
)istance: 399.00 cm | Level: 0.00% | SAFE
)istance: 100.00 cm | Level: 0.00% | SAFE
)istance: 399.00 cm | Level: 0.00% | SAFE
)istance: 399.00 cm | Level: 0.00% | SAFE
)istance: 398.00 cm | Level: 0.00% | SAFE
)istance: 399.00 cm | Level: 0.00% | SAFE
)ending SMS...
)MS Sent.
)istance: 7.00 cm | Level: 93.00% | DANGER
)istance: 10.00 cm | Level: 90.00% | DANGER
)istance: 100.00 cm | Level: 0.00% | SAFE
```